Checklist

# Engineering Executive:
# **How to Run Engineering Processes**

## Clarify Your Philosophy First

- [ ] Acknowledge that every process is a **trade-off between quality and overhead**
- [ ] Decide how much process sophistication your org can realistically sustain
- [ ] Avoid over-optimizing for perfection at the expense of usability
- [ ] Ensure processes are worth their operational cost

## Identify Your Current Pattern

Determine which pattern best matches your organization:

### Early Startup

- [ ] Founders or functional leaders run processes directly
- [ ] Processes are lightweight or ad hoc
- [ ] No dedicated support roles exist
- [ ] Work happens only if someone explicitly owns it

### Baseline (Most Common)

- [ ] Company-scoped processes run by centralized functions (e.g., People/HR)
- [ ] Engineering-scoped processes run by engineers/managers
- [ ] No specialized engineering process roles
- [ ] Engineering is treated similarly to other functions operationally

### Specialized Engineering Roles

- [ ] Dedicated EngOps / TPM / onboarding/incident roles exist
- [ ] Full-time support for engineering-scoped processes
- [ ] Process efficiency prioritized
- [ ] Additional headcount allocated for process support

### Company Embedded Roles

- [ ] Recruiting / People / Finance has dedicated Engineering partners
- [ ] Engineering processes diverge from company defaults
- [ ] Cross-functional embeds support Engineering directly

### Business Unit Local

- [ ] Engineering is split across business units
- [ ] No single engineering leader over the entire org
- [ ] Business units define their own processes
- [ ] Standardization across units is limited

## Evaluate Pros & Cons of Your Pattern

### Early Startup

- [ ] Low cost
- [ ] High flexibility
- [ ] Risk: Important processes don’t happen
- [ ] Risk: Quality depends heavily on the founders’ time/skill

### Baseline

- [ ] Clear ownership
- [ ] Modest specialization
- [ ] Unified systems across the company
- [ ] Risk: Outcomes depend on centralized function quality
- [ ] Risk: Hard to change company-wide processes

### Specialized Roles

- [ ] Increased efficiency

- [ ] Process expertise improves quality
- [ ] Engineers focus on engineering
- [ ] Higher cost (headcount)
- [ ] Risk of ossifying processes
- [ ] Specialists incentivized to improve processes, not eliminate them

### Company Embedded

- [ ] Engineering customization
- [ ] Stronger cross-functional alignment
- [ ] Expensive
- [ ] Quality dependent on embedded individuals

### Business Unit Local

- [ ] Strong business alignment
- [ ] Tailored trade-offs per unit
- [ ] Hard to standardize
- [ ] Cross-org investment decisions become difficult

## If You’re in Baseline (Recommended Default)

### Keep It Working Longer

- [ ] Don’t hire specialized roles too early (<100 engineers, reconsider carefully)
- [ ] Avoid copying patterns from much larger companies prematurely
- [ ] Improve execution before changing structure

### Make It Sustainable

- [ ] Rotate process ownership (e.g., 6-month rotations)
- [ ] Overlap transitions to preserve continuity
- [ ] Pair people in key process roles

### Make It Valued Work

- [ ] Expose process owners to strategic discussions
- [ ] Tie the process work to the promotion criteria

- [ ] Encourage engineers to serve in process roles before Staff+/management
- [ ] Publicly recognize process contributors

## Budget Reality Check

Before moving “up” the pattern curve:

- [ ] Confirm revenue or headcount is growing
- [ ] Expect resistance if the company is in cost-reduction mode
- [ ] Understand whether budgeting is:
    - [ ] Dollar-based
    - [ ] Headcount-based (harder to justify role swaps)
- [ ] Identify which department’s budget will absorb new roles
- [ ] Prepare for cross-executive pushback

Rule of thumb:

- [ ] Only move up the cost curve during growth phases
- [ ] Default to staying in Baseline unless scale clearly demands change

## Trend Discipline

- [ ] Avoid chasing process fads (Agile extremes, management swings, etc.)
- [ ] Ground processes in a clear set of beliefs
- [ ] Commit to a model for 3–4 years before major changes
- [ ] Make changes intentionally, not reactively

## Final Decision Filter

Before changing patterns, ask:

- [ ] Is the current model truly failing?
- [ ] Or are we under-executing within it?
- [ ] Are we solving a real-scale problem?

- [ ] Or reacting to perceived sophistication gaps?
- [ ] Can we sustain the operational overhead long term?

## Default Recommendation

If forced to choose without context:

- [ ] Start with or stay in **Baseline**
- [ ] Improve execution before increasing specialization
- [ ] Move up patterns only when the scale makes it unavoidable